基于iVerilog+GTKwave的轻量级仿真环境

Icarus Verilog (iverilog) 是一个轻量、开源、免费的 Verilog 编译器，支持全平台：Windows/Linux/MacOS。可以使用命令行编译 Verilog 文件，生成对应的仿真波形文件，并通过自带的 gtkwave 查看仿真波形。

安装

Windows

https://bleyer.org/icarus/ ，直接选择最新版本下载即可（应该已经整合了 gtkwave）。

Linux

运行以下两条指令即可：

```
sudo apt-get install iverilog
sudo apt-get install gtkwave
```

MacOS

```
brew install icarus-verilog
brew install gtkwave
```

注意：如果是MACOS 14，安装gtkwave后不能正常打开，先卸载之前的版本，再重新安装。

```
brew uninstall --cask gtkwave
brew install --HEAD randomplum/gtkwave/gtkwave
```

安装后，运行命令：

```
iverilog
```

若安装成功，运行命令后应出现类似如下的界面：

```
conjee@Yilis-MacBook-Pro Downloads % iverilog
iverilog: no source files.

Usage: iverilog [-EiRSuvV] [-B base] [-c cmdfile|-f cmdfile]
                [-g1995|-g2001|-g2005|-g2005-sv|-g2009|-g2012] [-g<feature>]
                [-D macro[=defn]] [-I includedir] [-L moduledir]
                [-M [mode=]depfile] [-m module]
                [-N file] [-o filename] [-p flag=value]
                [-s topmodule] [-t target] [-T min|typ|max]
                [-W class] [-y dir] [-Y suf] [-l file] source_file(s)

See the man page for details.
```

示例

我们以对一个 ALU 元件进行仿真为例，演示调试的基本流程。

首先，用任何你喜欢的文本编辑器编辑如下allu.v文件。

```
module alu(A,B,ALUOp,C);
    input [31:0] A,B;
    input [2:0] ALUOp;
```

```
    reg signed [31:0] ta,tb;
    output reg [31:0] C;
    always @(A or B or ALUOp) begin
        case (ALUOp)
            3'b000: C <= A + B;
            3'b001: C <= A - B;
            3'b010: C <= A & B;
            3'b011: C <= A | B;
            3'b100: C <= A >> B;
            3'b101: C <= $signed(A) >>> B;
            3'b110:
                if(A > B) C <= 1;
                else C <= 0;
            3'b111:
                if($signed(A) > $signed(B)) C <= 1;
                else C <= 0;
        endcase
    end
endmodule
```

仿真除了元件的代码以外，还需要一份 testbench 代码，为元件提供周期性的信号，并检查元件的输出是否符合预期。一个简单的 testbench如下，命名为tb.v。

```
`timescale 1ms/1ms
// testbench for simulation
module tb();

    reg [31:0] A,B;
    wire [31:0] C;
    reg [2:0] ALUOp;
    reg [10:0] cnt;
    // instantiation of ALU
    alu U_ALU(.A(A),.B(B),.ALUOp(ALUOp),.C(C));

    initial begin
        $dumpfile("test.vcd");// 指定生成的 vcd 文件名称
        $dumpvars;// 默认记录所有信号到 vcd 文件中
        $display("TEST");
        A=32'h00000003;
        B=32'h00000002;
        ALUOp=3'b000;
        cnt=10'b0;
        #(100) $finish;// 设置仿真停止时间
    end
    always begin
        #(5) ALUOp^=3'b001;
        cnt+=1;
    end //end always

endmodule
```

在 testbench 当中，我们首先实例化一个 ALU 元件，然后对其输入输出连线。
我们将输入信号 A 初始化为 3，输入信号 B 初始化为 2，ALUOp 初始化为 0，代表进行加法运算。
下面的 always 语句中，ALUOp 每过 5ms 从 0 变为 1，再过 5ms 从 1 变为 0。信号 C 的预期输出初始为 5，每过 5ms 变为 1，再过 5ms 变为1。
cnt 初始化为 0，预期每过 5ms 会加 1。

注意

testbench 中，以下代码为 iverilog 编译器专用，如果不加则无法生成波形文件。

```
initial begin
    $dumpfile("test.vcd");// 指定生成的 vcd 文件名称
    $dumpvars;// 默认记录所有信号到 vcd 文件中
end
```

并且仿真需要设定停止时间，否则可能会一直运行。

```
#(100) $finish;// 设置仿真停止时间，100时间单位后结束
```

编译与仿真

使用 iverilog 进行编译，命令如下：

```
iverilog -o test.out alu.v tb.v
```

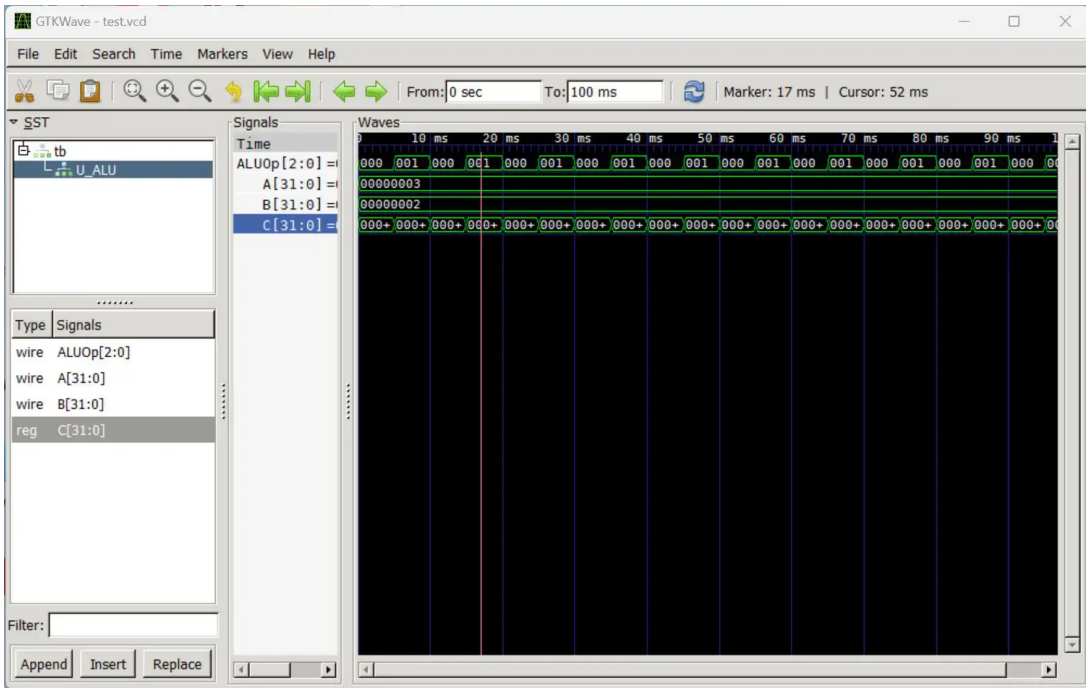
这段命令会编译文件 alu.v 与 tb.v，生成文件 test.out。
使用 vvp 命令生成波形文件 test.vcd：

```
vvp test.out
```

使用 gtkwave 查看波形文件：

```
gtkwave test.vcd
```

打开如下界面：



观察可以发现 C 的输出初始为 5，经过 5ms 后变为 1，再经过 5ms 后变为 5，与预期相符。